Continuous Integration with MATLAB

From local tests to a repeatable GitHub build

25 min

Concepts and configuration

30 min

MATLAB + GitHub demo

5 min

Questions and next steps

A change should come with evidence

The change

A colleague updates a function.
What else could it affect?

The uncertainty

Different files, paths, inputs, and environments.

The evidence

A repeatable build and readable test results.

Get feedback while the change is still small.

VOCABULARY

CI and CD describe different steps

Continuous integration

Integrate changes often.
Build and test automatically.

Continuous delivery

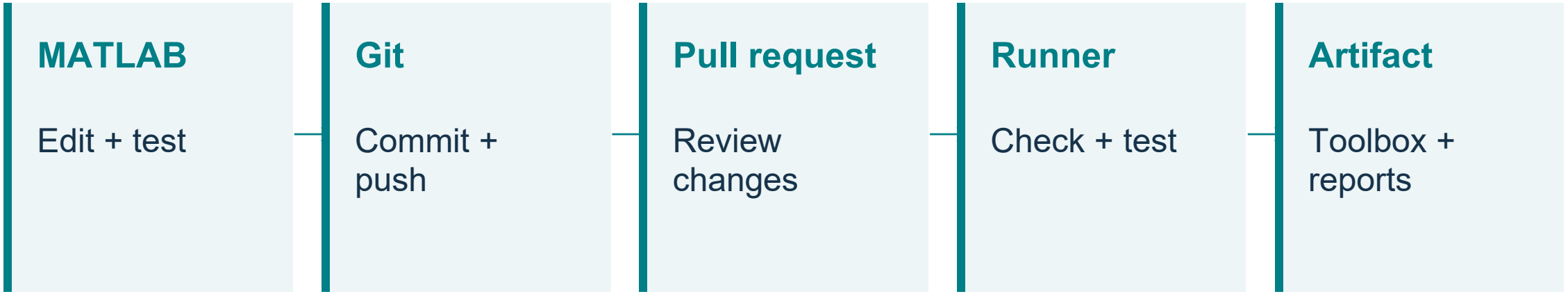
Keep validated changes ready for release.
A person decides when.

Continuous deployment

Deploy successful changes to the target environment automatically.

Today: CI + a downloadable toolbox artifact

Follow one change through the system



A failed test stops the package task. A passing build supplies evidence for review.

Git records changes. Actions runs checks.

In MATLAB

Branch → edit → compare → commit
Push commits to GitHub
Pull merged changes back

In GitHub

Pull request → checks → review → merge
Actions chooses a runner
The runner executes the workflow

A workflow is a YAML file stored with the code.

A test states expected behavior

```
function firstDayOfYear(testCase)
    dateText = "2026-01-01";       % Arrange
    actual = dayofyear(dateText); % Act
    testCase.verifyEqual(actual,1); % Verify
end
```

Known answer

Start with a result you can explain.

Boundary case

Test where behavior changes.

Coverage

Find gaps in execution; keep thinking about inputs.

Three files, three responsibilities

matlab.toml

What belongs to the project?
Paths, tests, dependencies, package metadata.

buildfile.m

What must happen?
Check → test → package.
Task dependencies in MATLAB.

ci.yml

When and where?
Events, runner, release, actions, and uploaded outputs.

```
buildtool package -report artifacts/build-report
```

A readable project definition

```
name = "CI Seminar"

[folders]
path = ["src"]
test = ["tests"]
```

Project context

Same source path.
Same test discovery.
Readable changes in Git.

TOML configures the MATLAB project; YAML configures the GitHub workflow.

Read YAML from the outside in

```yaml
name: MATLAB CI
on:
  pull_request:
    branches: [main]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: matlab-actions/setup-matlab@v3
        with:
          release: R2026b
      - uses: matlab-actions/run-build@v3
        with:
          tasks: package
```

WHEN
An event starts the workflow

WHERE
A fresh GitHub-hosted runner

WHAT
Checkout → MATLAB → build

What R2026b adds to this workflow

Project and collaboration

matlab.toml project definitions
Declared project test folders
Connections + Pull Requests panels

Build and feedback

Built-in PackageTask → .mltbx
HTML build reports
Task output caching

Also new: runtests(...,UseTestBrowser=true)

Established: Projects, unit tests, Git, and buildtool (introduced in R2022b).

Choose the runner and check licensing

Hosted runner

Clean environment for each job.
Our demo: public repo + Ubuntu
+ explicitly selected R2026b.

Self-hosted runner

Your configured machine.
Existing MATLAB installation.
Your licensing and access controls.

Public base-MATLAB demo: automatic licensing under the documented policy.

Private repositories and transformation products need additional arrangements.

Live demo: make the checks tell a story

1 Start green

Project + three tests
Same local and CI build

2 Find a gap

March 1, 1900
Expected day: 60

3 Fix and deliver

Five tests pass
Download the toolbox

```
buildtool package -report artifacts/build-report
```

Compare • commit • push • pull request • checks • merge • pull

QUESTIONS

Start with one useful test and one workflow

Write the expectation

Use tests to describe what must remain true.

Make it repeatable

Commit the project and the build plan.

Automate the feedback

Run the same checks for every pull request.

Resources

Demo: github.com/kevinholly/matlab-ci-seminar
mathworks/ci-configuration-examples
mathworks/ci-with-matlab-and-github-actions-workshop

APPENDIX A

The smallest useful test workflow

```yaml
name: MATLAB tests
on:
  pull_request:
    branches: [main]
  workflow_dispatch:
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: matlab-actions/setup-matlab@v3
        with:
          release: R2026b
      - uses: matlab-actions/run-tests@v3
        with:
          source-folder: src
          select-by-folder: tests
          code-coverage-metrics: statement
```

Keep reports when a build fails

```yaml
- name: Check, test, and package
  uses: matlab-actions/run-build@v3
  with:
    tasks: package
    build-options: -report artifacts/build-report

- name: Upload reports
  if: always()
  uses: actions/upload-artifact@v7
  with:
    name: matlab-ci-reports
    path: artifacts/

- name: Upload toolbox
  uses: actions/upload-artifact@v7
  with:
    name: calendar-utilities-toolbox
    path: release/*.mltbx
```

MATLAB owns the task dependencies

```
function plan = buildfile
import matlab.buildtool.tasks.*
plan = buildplan;
plan("check") = CodeIssuesTask("src", ...
    Configuration="factory",ErrorThreshold=0);
testTask = TestTask(Dependencies="check", ...
    SourceFiles="src", ...
    TestResults=["artifacts/tests/results.xml", ...
                 "artifacts/tests/report.html"]);
testTask = addCodeCoverage(testTask, ...
    "artifacts/coverage/report.html",Metrics="statement");
plan("test") = testTask;
plan("package") = PackageTask(Dependencies="test");
plan.DefaultTasks = "package";
end
```

A boundary case changes the requirement

```
% New expectations
testCase.verifyEqual(dayofyear("1900-03-01"),60);
testCase.verifyEqual(dayofyear("2000-03-01"),61);

% Complete Gregorian leap-year rule
if mod(dateValue.Year,4) == 0 && ...
        (mod(dateValue.Year,100) ~= 0 || ...
         mod(dateValue.Year,400) == 0)
    daysPerMonth(2) = 29;
end
```

A green suite only covers the expectations you have actually tested.

APPENDIX E

References and extensions

Start here

MathWorks CI examples
GitHub Actions workshop
setup-matlab / run-tests / run-build

Go further

R2026b release notes
TOML project reference
Build tool and PackageTask

Full source URLs are in the speaker notes and companion Markdown.